

MIDTERM · PAPER PRESENTATION

Learning to Route

Neural Combinatorial Optimization with Reinforcement Learning

Bello, Pham, Le, Norouzi, Bengio · Google Brain · arXiv 1611.09940

Saina Zardosht

Shazan Ansar Mohammed

CSC541-Dr. Prof. Bin Tang



THE PROBLEM

One van. 20 stops. How many routes?

60,822,550,204,416,000

possible round trips through 20 stops — $(n - 1)! / 2$

4.7×10^{155}

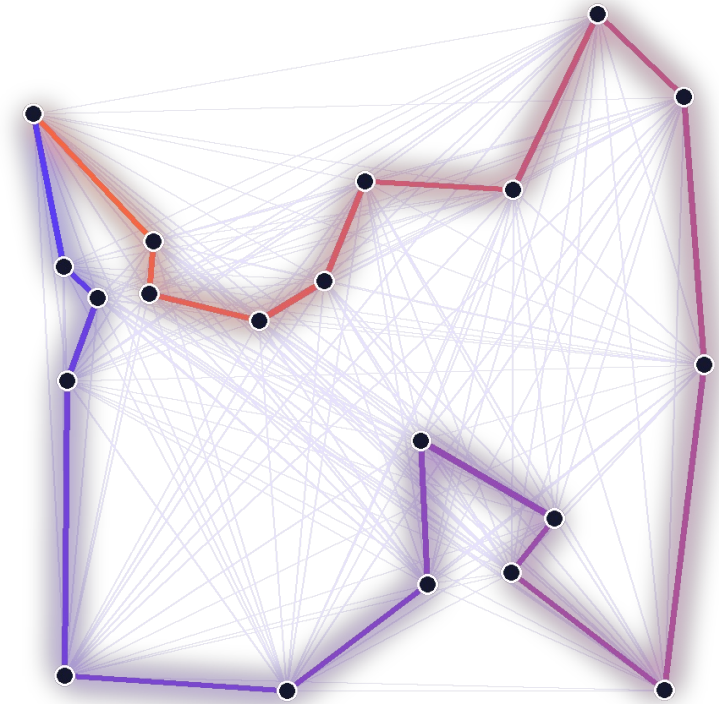
routes for 100 stops

$\sim 10^{80}$

atoms in the observable
universe

NP-hard

even on a flat 2D map



At right: every faint line is a possible leg. The bright path is the one we want.

TODAY

Exact DP: $\Theta(2^n n^2)$

The best solvers are hand-engineered



EXACT

Concorde

Cutting planes + branch-and-bound

Proves the optimal tour



HEURISTIC

LKH

Lin–Kernighan–Helsgaun local moves

Hundreds of nodes solved optimally



GENERAL

OR-Tools

Google's local search + metaheuristics

Handles many routing variants

Change the problem slightly → the heuristics must be rewritten

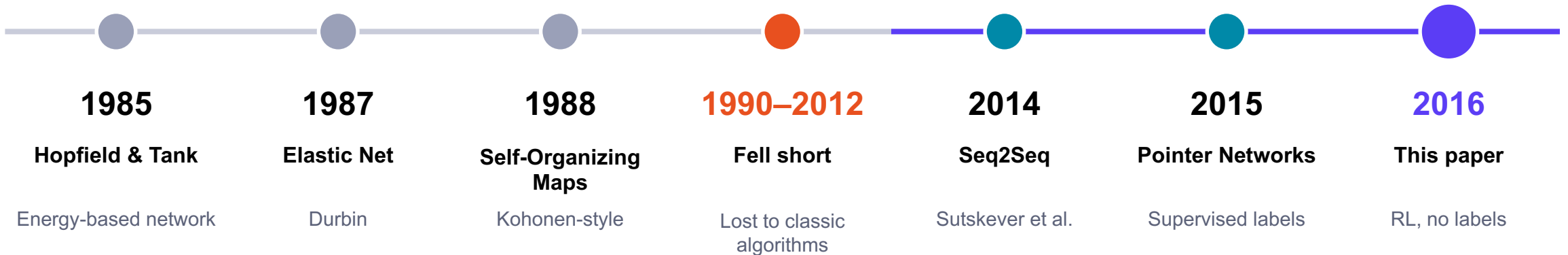


CONTEXT

Neural networks have chased TSP since 1985

CLASSIC NEURAL ERA

DEEP LEARNING ERA

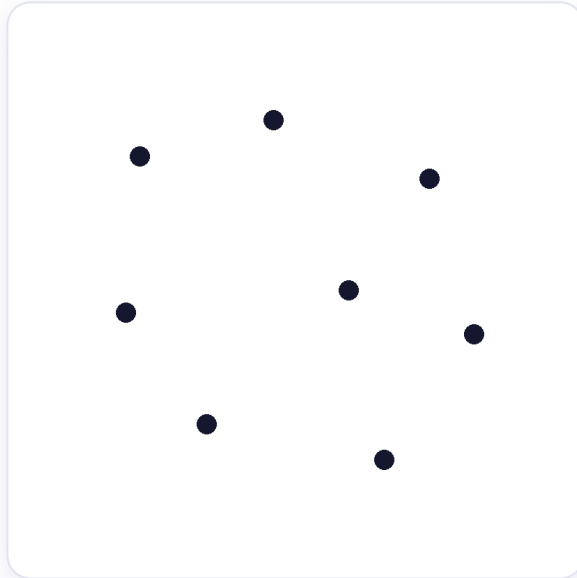


What changed in 2016: sequence models + learning from reward instead of labels.

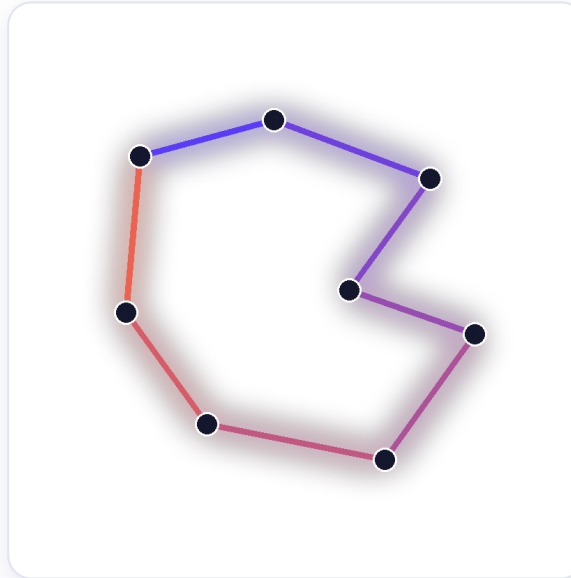
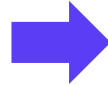


PROBLEM STATEMENT

Input: points. Output: an order



cities $s = \{x_1 \dots x_n\} \subset [0, 1]^2$



tour π : visit each once, return

MINIMIZE THE TOUR LENGTH

$$L(\pi | s) = \|x_{\pi(n)} - x_{\pi(1)}\|_2 + \sum_{i=1}^{n-1} \|x_{\pi(i)} - x_{\pi(i+1)}\|_2$$

LEARN A POLICY, ONE CITY AT A TIME

$$p(\pi | s) = \prod_{i=1}^n p(\pi(i) | \pi(< i), s)$$

2D Euclidean

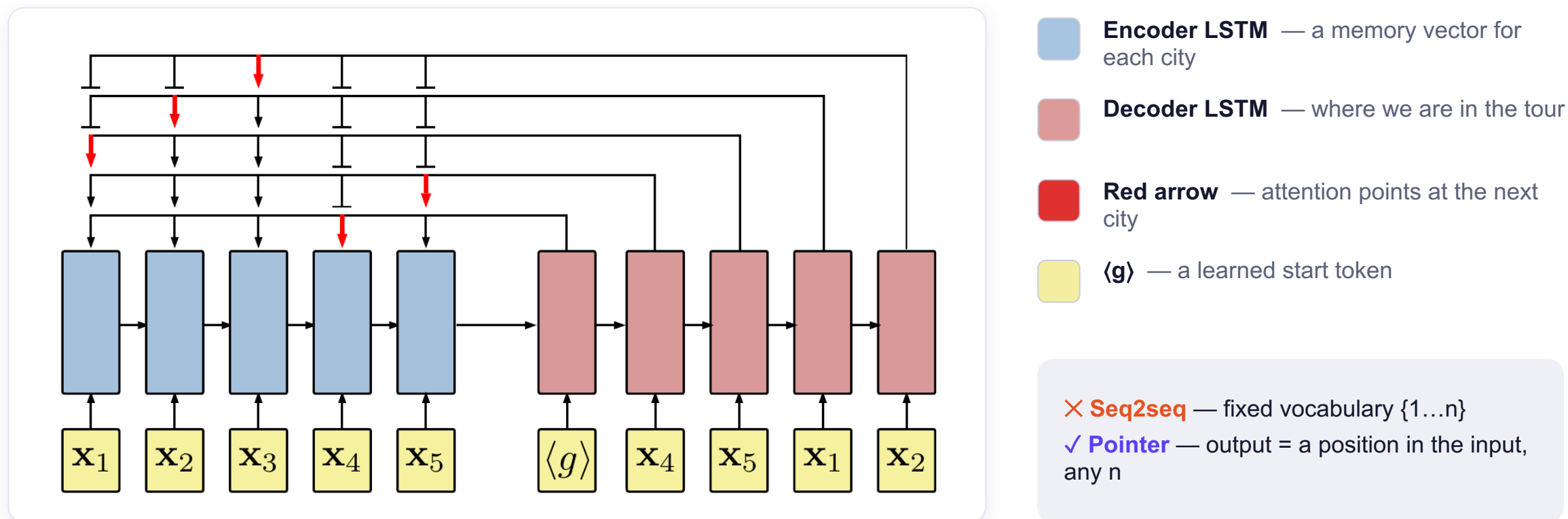
uniform in $[0, 1]^2$

$n = 20 \cdot 50 \cdot 100$

THE MODEL · POINTER NETWORK

Read every city. Then point to the next one

FROM THE PAPER · FIGURE 1



ATTENTION = POINTING

The attention scores are the output

- 1 Score every city against the decoder state

$$u_i = v^\top \tanh(W_{ref} e_i + W_q d_t)$$

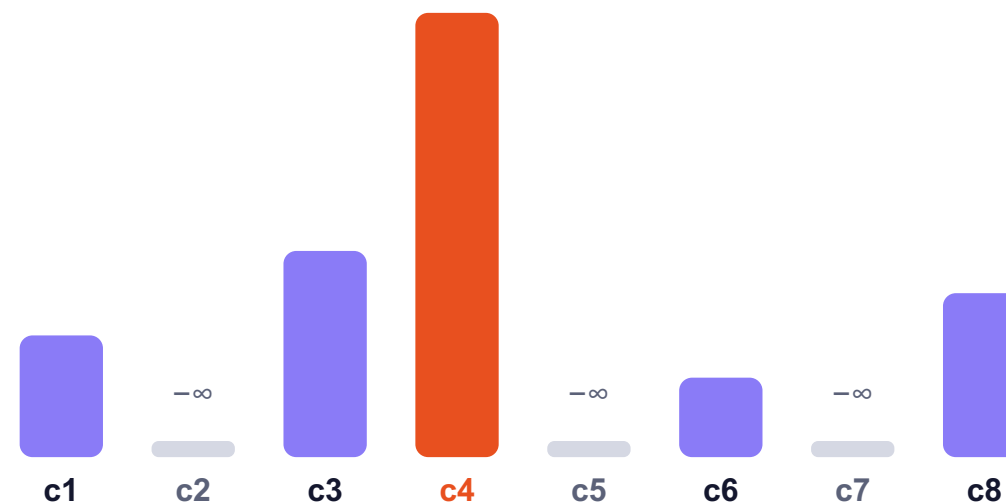
- 2 Mask cities already in the tour

$$u_i = -\infty \quad \text{if city } i \text{ already visited}$$

- 3 Normalize with clipped logits, then pick

$$p(\text{next} = i) = \text{softmax}(C \tanh(u))_i$$

ONE DECODING STEP · ILLUSTRATIVE



▲ Next city = c4, the highest probability

Mask → always a valid tour

1 glimpse helps · more don't

Logit clipping C = 10 aids exploration

No labels needed. Just measure the tour



POLICY GRADIENT (REINFORCE)

$$\nabla_{\theta} J = \mathbb{E}_{\pi \sim p_{\theta}} [(L(\pi | s) - b(s)) \nabla_{\theta} \log p_{\theta}(\pi | s)]$$

$b(s)$ comes from a critic network: a per-graph baseline that cuts gradient variance.

Supervised imitation is capped by its teacher: **39% gap on TSP100.**

FROM THE PAPER · ALGORITHM 1

Algorithm 1 Actor-critic training

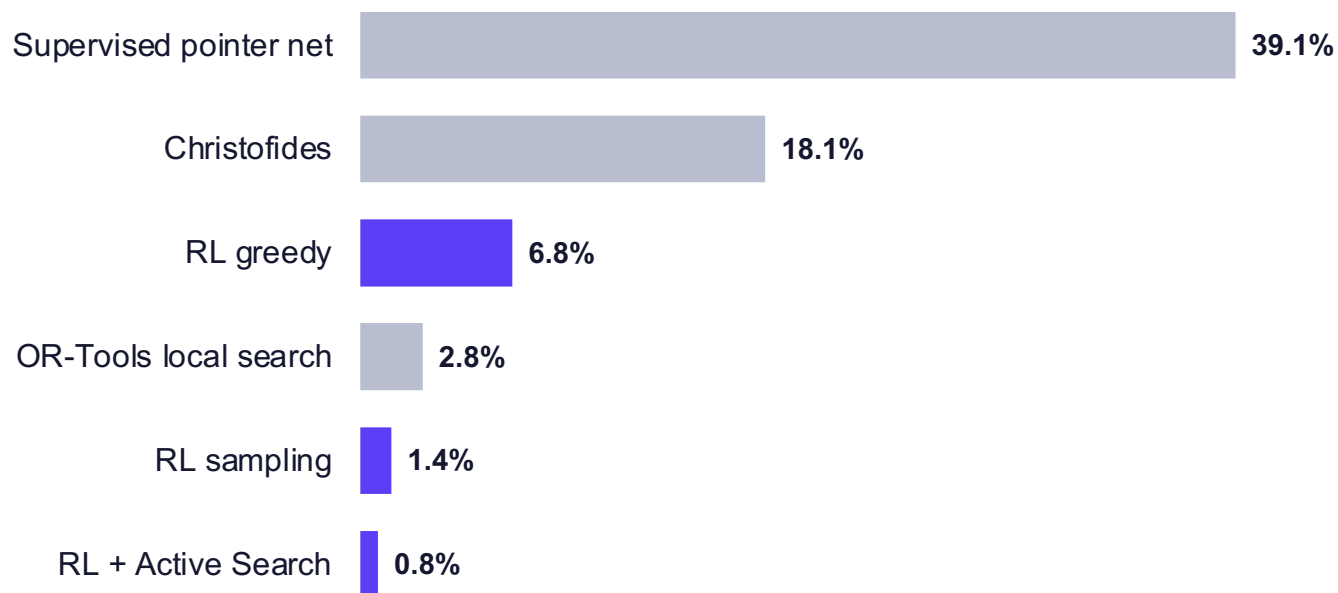
```

1: procedure TRAIN(training set  $S$ , number of training steps  $T$ , batch size  $B$ )
2:   Initialize pointer network params  $\theta$ 
3:   Initialize critic network params  $\theta_v$ 
4:   for  $t = 1$  to  $T$  do
5:      $s_i \sim \text{SAMPLEINPUT}(S)$  for  $i \in \{1, \dots, B\}$ 
6:      $\pi_i \sim \text{SAMPLESOLUTION}(p_{\theta}(\cdot | s_i))$  for  $i \in \{1, \dots, B\}$ 
7:      $b_i \leftarrow b_{\theta_v}(s_i)$  for  $i \in \{1, \dots, B\}$ 
8:      $g_{\theta} \leftarrow \frac{1}{B} \sum_{i=1}^B (L(\pi_i | s_i) - b_i) \nabla_{\theta} \log p_{\theta}(\pi_i | s_i)$ 
9:      $\mathcal{L}_v \leftarrow \frac{1}{B} \sum_{i=1}^B \|b_i - L(\pi_i)\|_2^2$ 
10:     $\theta \leftarrow \text{ADAM}(\theta, g_{\theta})$ 
11:     $\theta_v \leftarrow \text{ADAM}(\theta_v, \nabla_{\theta_v} \mathcal{L}_v)$ 
12:  end for
13:  return  $\theta$ 
14: end procedure
  
```

WHAT THE PAPER FOUND

Near-optimal on 100 cities, and fast

TSP100 · GAP TO OPTIMAL (LOWER IS BETTER)



RL (this paper)

Baselines

 **22x**

faster than Concorde

RL greedy on TSP100: 0.01 s vs 0.22 s per graph

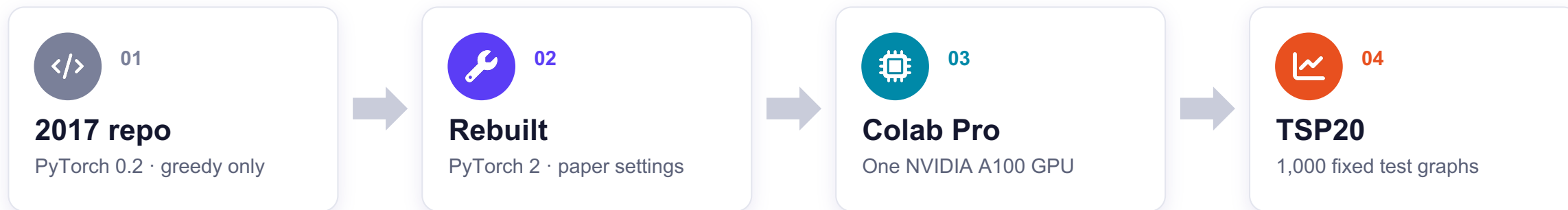
 **200**

items solved optimally

KnapSack, by Active Search (Table 5)

OUR EXPERIMENT

Can we reproduce it on one GPU?



THE PAPER'S CHOICE

Critic network

Learns a baseline $b(s)$ for each graph

vs

THE REPO'S CHOICE

Moving average

One running mean shared by all graphs

20k steps · ~20 min each

batch 512

lr 1e-3

128-d LSTM

1 glimpse

sampling $\times 1280$ · $T = 1, 2$

vs NN · 2-opt · LKH

OUR RESULTS

In our run, the paper's critic halves the gap

4.6%

gap to optimal · greedy

critic · vs 9.0% with the moving average

1.6%

with sampling ×1280

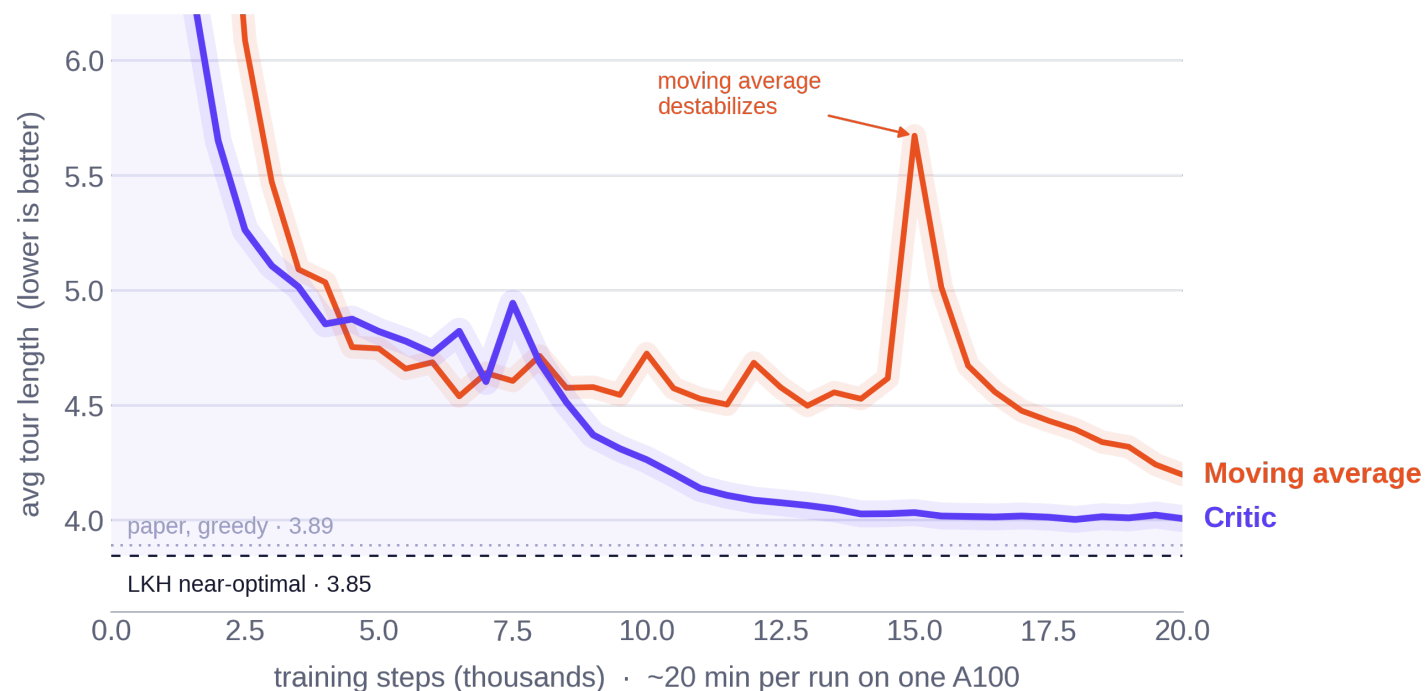
matches LKH exactly on 29% of graphs

20 min

training per run

one NVIDIA A100 · 20k steps

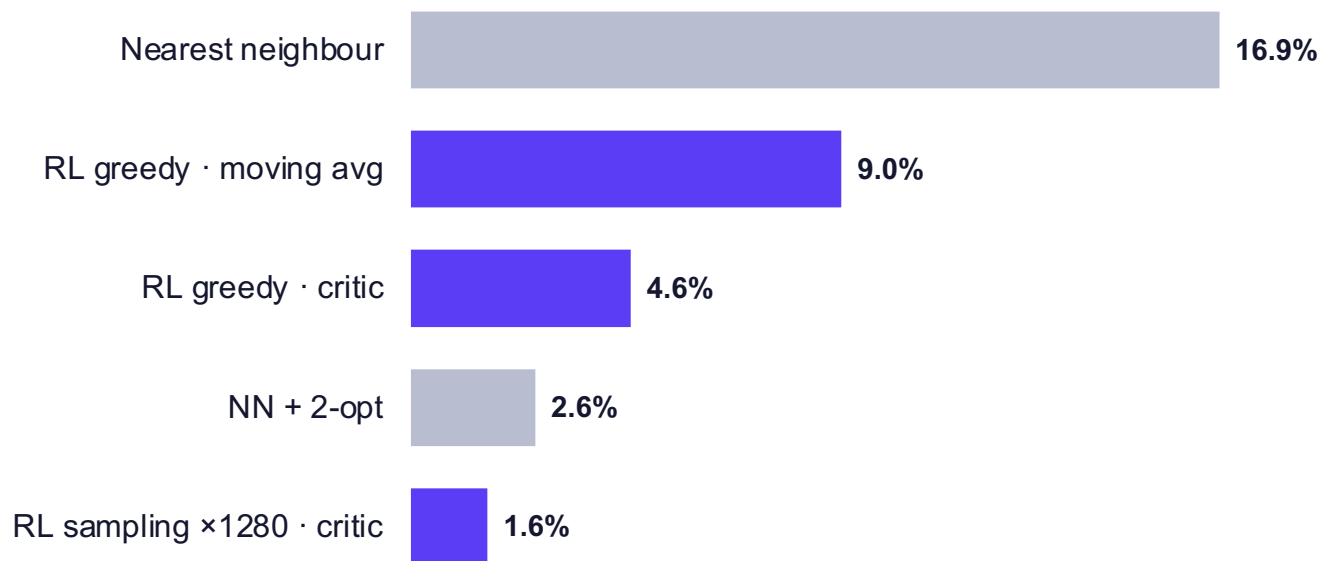
VALIDATION TOUR LENGTH DURING TRAINING



WHERE WE STAND

With sampling, our model beats classic 2-opt

TSP20 · GAP TO LKH (LOWER IS BETTER)

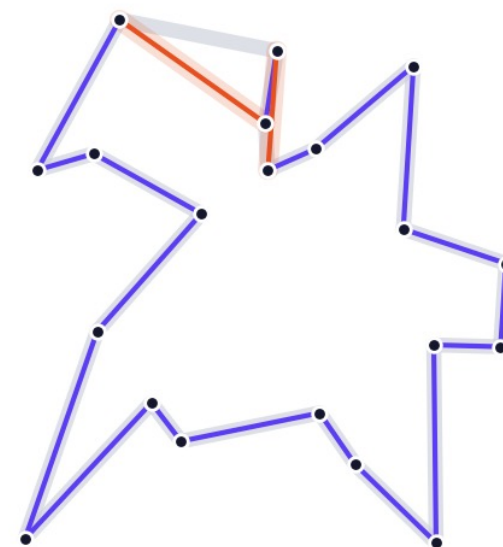


Ours

Classic

A TYPICAL GRAPH · OURS VS LKH

Ours 4.524 vs LKH 4.356 · 2 of 20 legs differ



Orange = legs that differ from LKH. 2 wrong legs cost 3.9%.

WHAT'S NEXT

From reproduction to research



Thank you

Questions? · Saina Zardosht & Shazan Ansar Mohammed

Github <https://github.com/SHAZAN01/neural-combinatorial-rl-tsp>

Colab https://colab.research.google.com/drive/13ov5Xa0ejhLAG6gyGUhRDHUEBBiUquWA#scrollTo=5LDa1_0k1OVu

Full results from the paper

TABLE 2 · AVERAGE TOUR LENGTH (LOWER IS BETTER)

Task	Supervised	RL greedy	greedy@16	RL sampling	RL + AS	Active Search	Christofides	OR-Tools	Optimal
TSP20	3.88†	3.89	–	3.82	3.82	3.96	4.30	3.85	3.82
TSP50	6.09†	5.95	5.80	5.70	5.70	5.87	6.62	5.80	5.68
TSP100	10.81	8.30	7.97	7.88	7.83	8.19	9.18	7.99	7.77

TABLE 3 · RUNNING TIME PER GRAPH

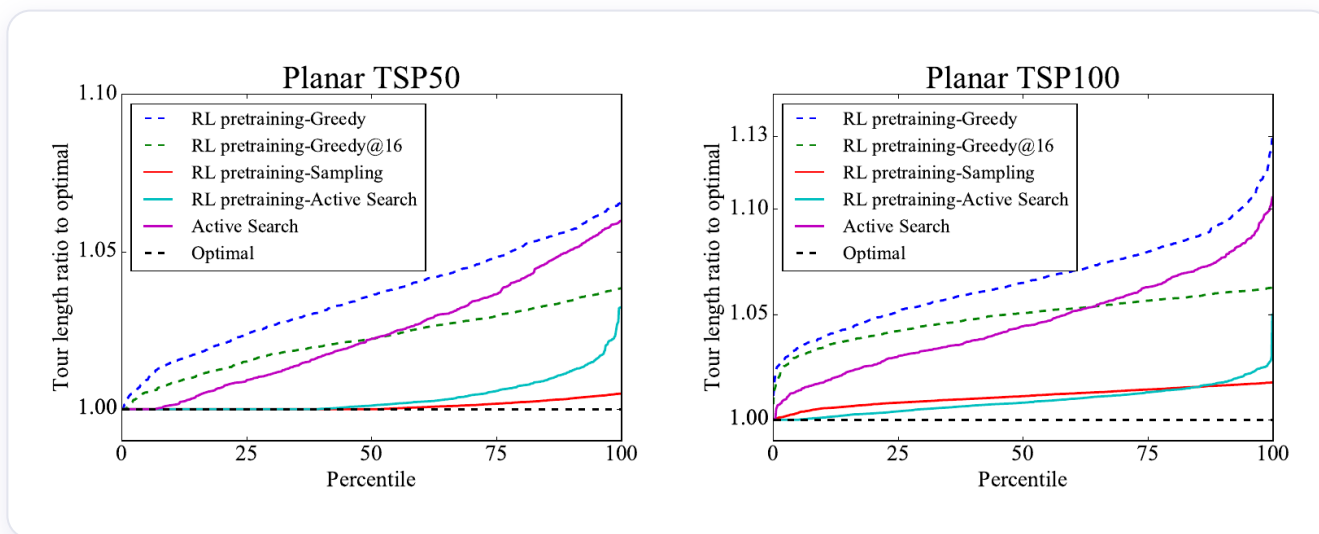
Task	RL greedy	RL greedy@16	OR-Tools local search	Concorde	LK-H
TSP50	0.003 s	0.04 s	0.02 s	0.05 s	0.14 s
TSP100	0.01 s	0.15 s	0.10 s	0.22 s	0.88 s

Source: Bello et al., 2017 · † from Vinyals et al., 2015 · AS = Active Search

BACKUP

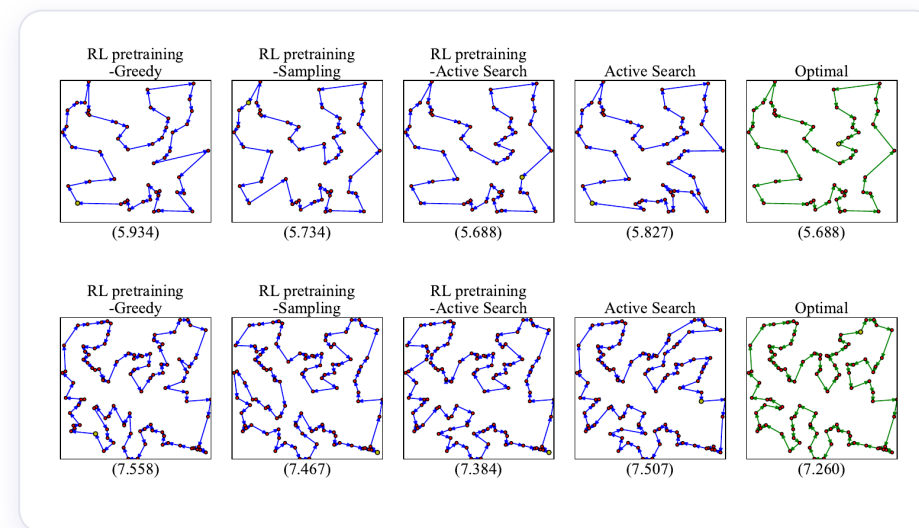
How close each method gets

FIGURE 2 · SORTED RATIO TO OPTIMAL



RL + Active Search stays close to the optimal line on most TSP100 graphs.

FIGURE 3 · SAMPLE TOURS

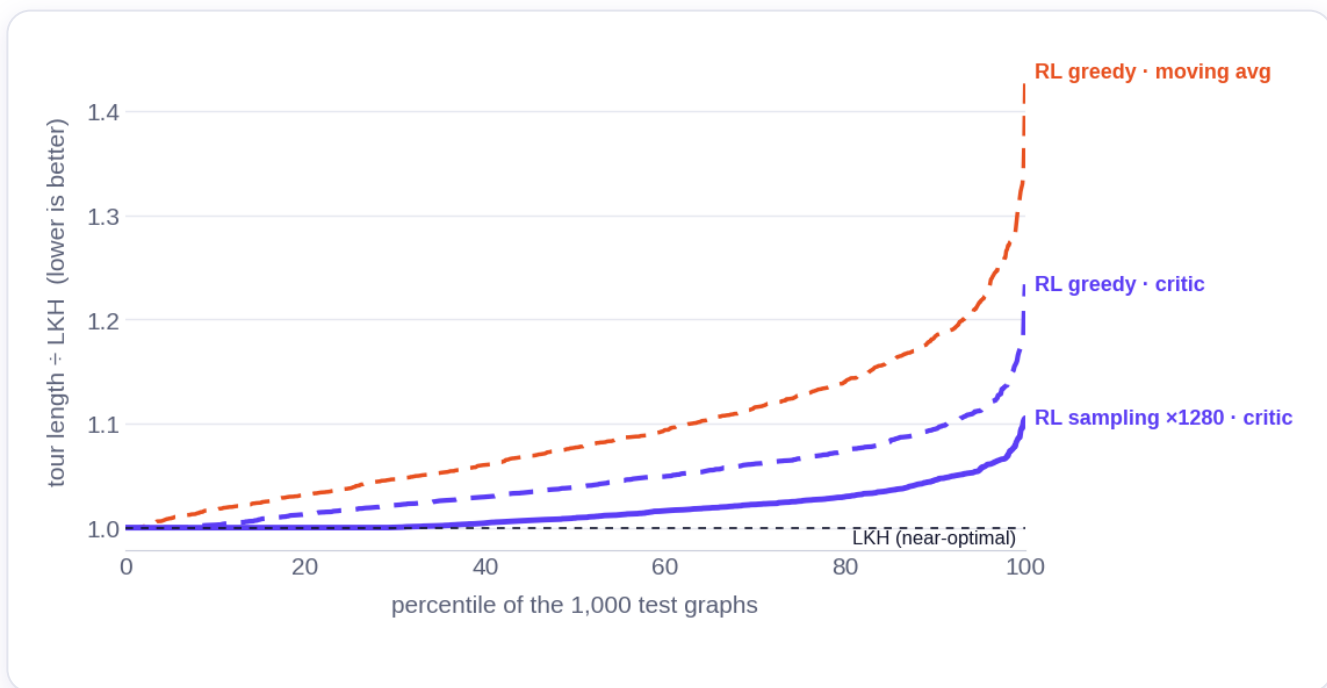


Sample TSP100 tour: 7.558 greedy → 7.384 RL + AS, vs 7.260 optimal.

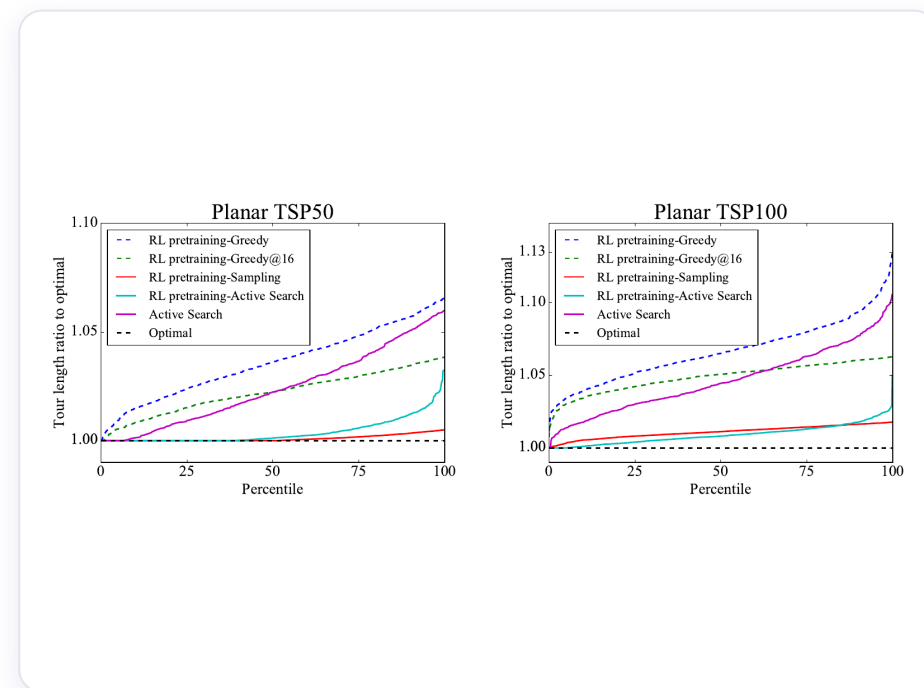
BACKUP

Our Figure 2, next to the paper's

OURS · TSP20 · 1,000 TEST GRAPHS VS LKH



PAPER · TSP50 / TSP100 VS OPTIMAL



Same shape as the paper: sampling hugs the optimal line; greedy drifts up; the moving average is worst.

Ours: median gap 3.9% greedy (critic), 0.9% sampling · paper: Bello et al., 2017, Fig. 2

BACKUP

Speed vs. quality



How to read it

Lower-left is better. The dashed line joins the methods nothing else beats on both speed and quality.

Caveat

RL ran batched on a GPU, classic methods on a CPU, so the time axis is not hardware-matched.

Times per graph from our Colab run (1,000 TSP20 graphs)